READY

> TECHNICAL-DEEP-DIVE.SPARK

Apache Spark

Lazy Evaluation & Partitioning

Liza Malinina — Senior Data Engineer, Microsoft

Liza Malinina

- → Senior Data Engineer @ Microsoft
- → Data Architect & Co-Founder @ MalenStudio
- → 10+ years in industry
- → Live in Estonia
- → Have a 3-year-old daughter (you'll see why it matters)

Agenda

- → What is Apache Spark?
- → Lazy Evaluations
  - ↳ Demo 1
- → Partitioning
  - ↳ Demo 2
- → Caching, Pitfalls & Debugging
  - ↳ Demo 3
- → Q & A

Apache Spark™

A multi-language engine for executing data engineering, data science, and machine learning on single-node machines or clusters.

Supported Languages:

Apache Spark: Three Pillars

1. Distributed Computing

Processes data in parallel across a cluster of machines

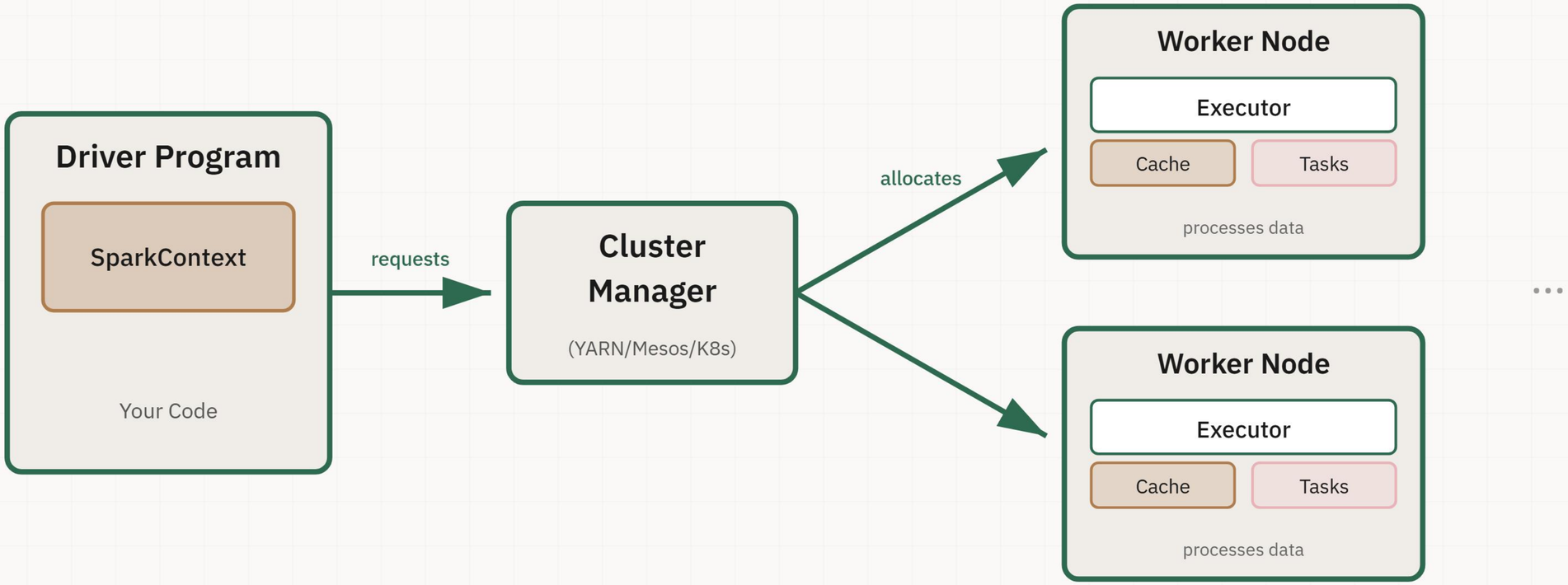
2. DataFrame API + DAG

Represents transformations as a DAG instead of running line-by-line

3. High Performance

Keeps data in memory between operations

Driver / Executors Architecture



Problem Statement

```
df.filter().groupBy().withColumn().count()
```

→ "Why did Spark run 4 shuffles?"

→ "Why did this stage take 17 minutes?"

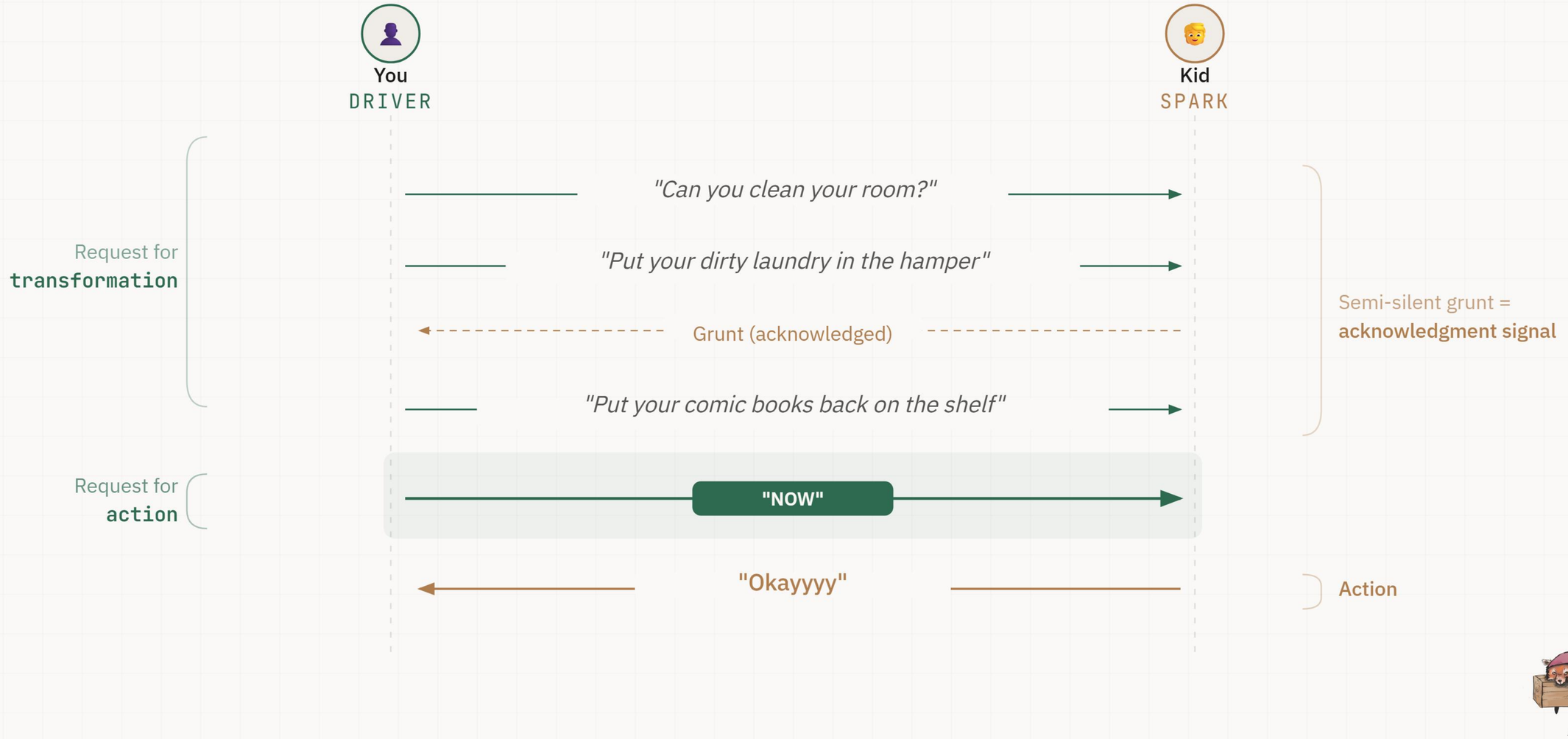
→ "Why are there 17k output files?"

Ususally, the problem traced back to one of two things: **lazy evaluation** or **partitioning**.

PART 1

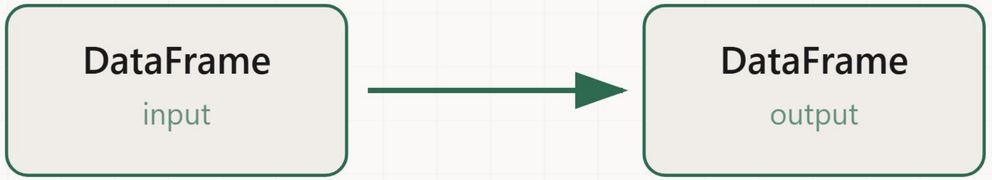
Lazy Evaluations

The Toddler and the Right Tone



Transformations

Transformations — operations that return another DataFrame.

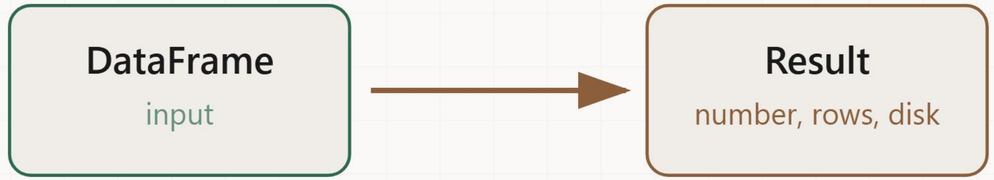


Examples: `filter()` , `select()` , `groupBy()` , `withColumn()` , `join()`

Lazy. Spark writes down the recipe but doesn't cook.

Actions

Actions — operations that produce a result or write data.



Examples: `count()` , `show()` , `collect()` , `write()`

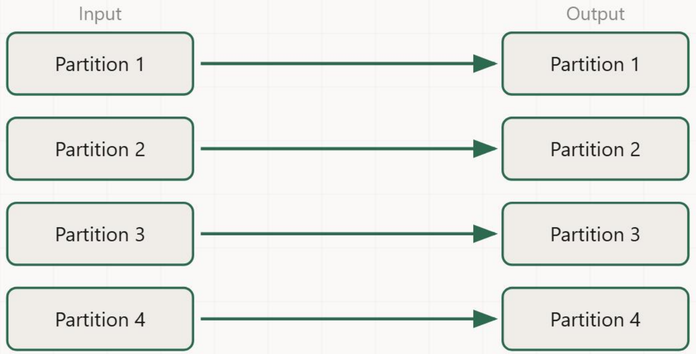
Eager. This is "NOW." Spark executes everything.

Narrow vs Wide Transformations

Narrow

Each input partition → at most one output partition

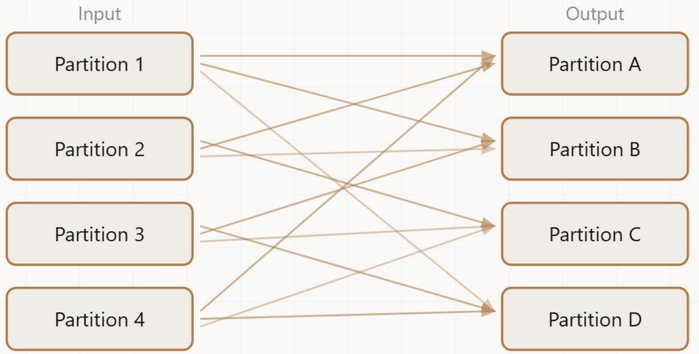
`filter()` , `select()` , `map()` , `withColumn()`



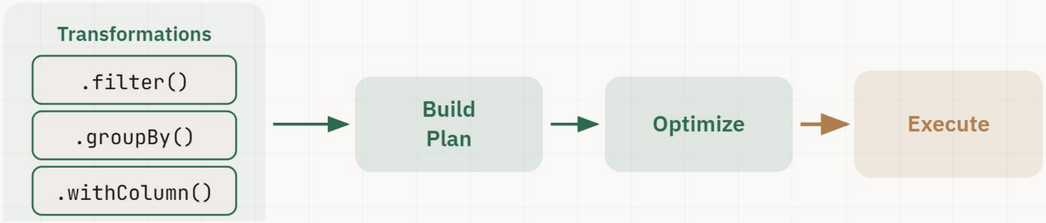
Wide (SHUFFLE)

Data must move between partitions

`groupBy()` , `join()` , `orderBy()` , `repartition()`



How Lazy Evaluation Works



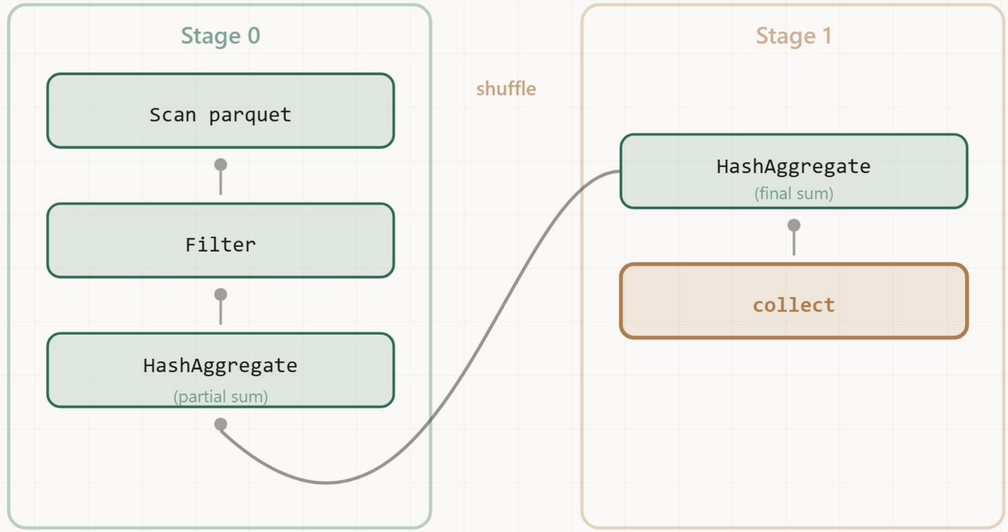
1. **Avoids unnecessary work** — never reads data you'll filter out
2. **Enables optimization** — combine filters, prune columns
3. **Minimizes data movement** — fewer shuffles

DAG: The Blueprint

→ **D**irected — data flows one way

→ **A**cyclic — no loops

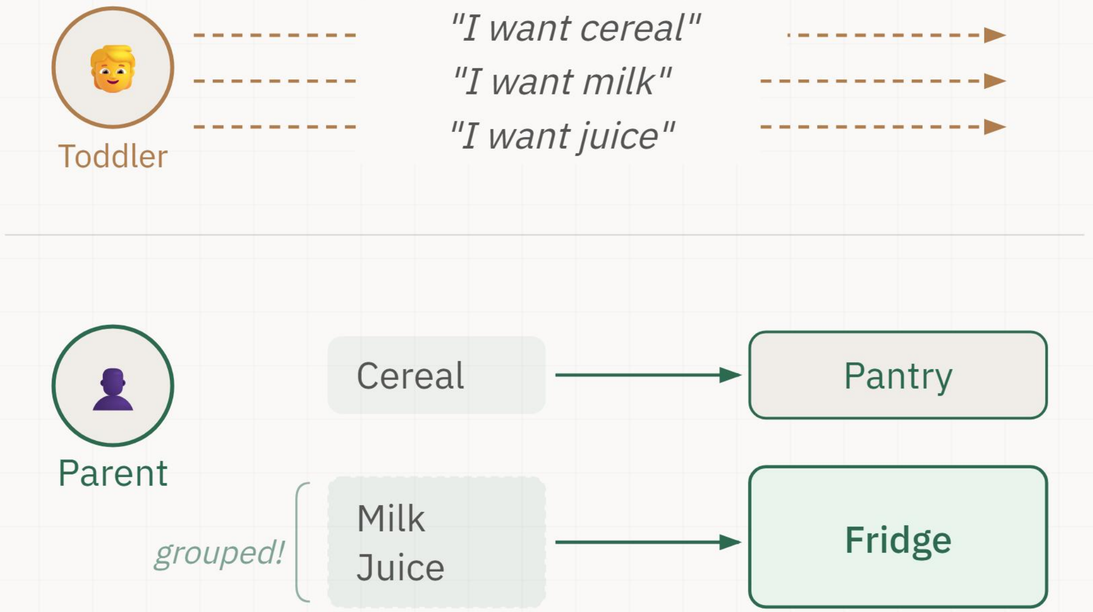
→ **G**raph — operations connected by data flows



Catalyst Optimizer

Catalyst turns your plan into a better plan:

1. **Logical Plan** — resolve column references, verify types
2. **Optimized Logical Plan** — predicate pushdown, column pruning, filter combining
3. **Physical Plan** — choose execution strategy for the cluster



Catalyst: The Proof

Experiment	What Happens	Time
Load + collect	Nothing extra	~21 sec
Load + create 3 columns + collect	3 withColumn()	~34 sec
Load + create 3 columns + **drop them** + collect	3 withColumn + 3 drop	**~25 sec**

Creating AND deleting columns is **faster** than just creating them?

Catalyst saw the full plan and skipped the unnecessary work entirely.

Source: Spark in Action (Manning)

DEMO 1

Let's see it live

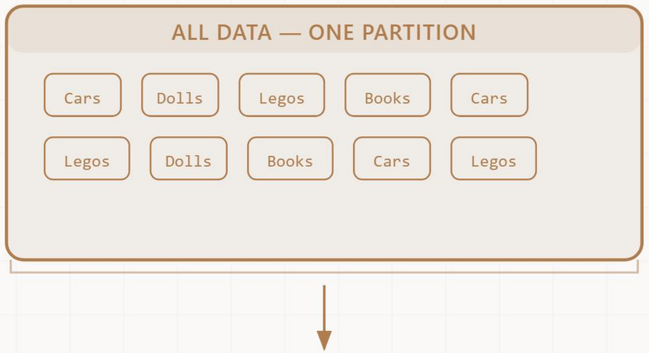
Lazy evaluation · Catalyst optimizer · reading execution plans

PART 2

Partitioning

Where are the Legos?

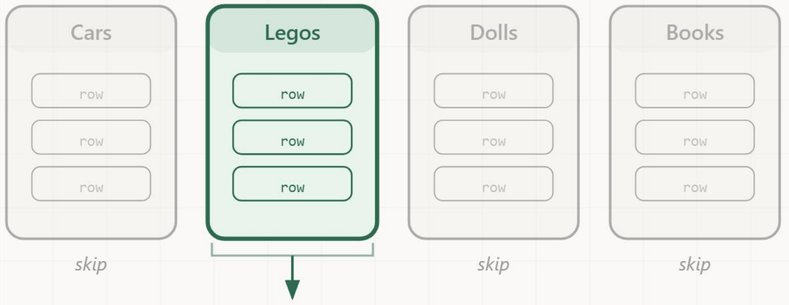
Without partitioning



Scan every record to find Legos
full table scan

Reads 100% of data

With partitioning



Go straight to the Legos partition
partition pruning

Reads only what you need

Two Levels of Partitioning

	In-Memory	On-Disk
What	How Spark splits data across executors while processing	How data is physically organized in storage
Control	`spark.sql.shuffle.partitions` (default: 200)	`df.write.partitionBy("year", "month")`
Key Point	Each partition = one task = one core	Enables **partition pruning** — Spark skips irrelevant folders entirely
Example	200 partitions processing in parallel	Folder structure: `year=2024/month=01/`

Filtering by Date

Scenario	Data Read	Time
No partitioning — full scan	100 GB	~10 min
Partitioned by date — query one month	8 GB	~50 sec
+ Parquet + predicate pushdown	0.5 GB	~5 sec

** illustrative estimates*

Lazy evaluation decides **when** to optimize.

Partitioning gives it **what** to optimize.

Together, that's the difference between 10 minutes and 5 seconds.

Partitioning Pitfalls & Fixes

Too few partitions

Cluster underutilized

Fix: `repartition(n)` — full shuffle, even distribution

Too many partitions

Small files problem, scheduling overhead

Fix: `coalesce(n)` — no shuffle, only decreases

Skewed partitions

One executor drowning, others idle

Fix: Salting, Adaptive Query Execution (AQE in Spark 3.0+)

Rule of thumb: `repartition` before heavy computation. `coalesce` before writing output.

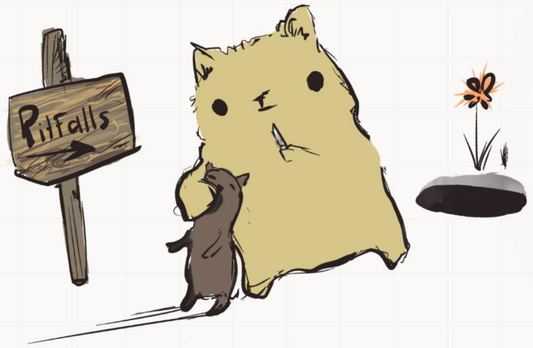
DEMO 2

Let's see it live

On-disk partitioning · partition pruning · repartition vs coalesce · shuffle tuning

PART 3

Caching, Pitfalls & Debugging



Problem 1 of 3

Repeated Actions

```
count = df.filter(...).count()                # full scan
revenue = df.filter(...).agg(...).collect()  # full scan AGAIN
top5 = df.filter(...).orderBy(...).take(5)   # full scan AGAIN
```

- Each action triggers a complete re-execution of the entire plan.
- The filter runs three times.
- The data read happens three times.
- Spark doesn't cache results between actions automatically.

Fix for Problem 1

Repeated Actions → cache() / persist()

```
filtered = df.filter(...)
filtered.cache()

count = filtered.count()                # one scan
revenue = filtered.agg(...).collect() # reuses cache
top5 = filtered.orderBy(...).take(5)  # reuses cache
```

- Stores the result in memory. Subsequent actions reuse it.
- Lineage is kept — if a cached partition gets evicted, Spark can recompute it from the source.

Problem 2 of 3

Lineage Explosion

A 50-stage job fails at stage 48.

Spark must recompute **all 48 stages** — the lineage, the full recipe, goes all the way back to the source.

On a noisy or overloaded cluster, this can be catastrophic.

Fix for Problem 2

Lineage Explosion → checkpoint()

```
spark.sparkContext.setCheckpointDir("/checkpoints")

df_mid = df.stage1().stage2()...stage25()
df_mid.checkpoint()  # saves to disk, breaks lineage

df_final = df_mid.stage26()...stage50()
```

- Saves progress to reliable storage and **breaks the lineage**.
- A failure at stage 30 restarts from the checkpoint file, not from zero.

Problem 3 of 3

Debugging Confusion

Error surfaces at `.collect()`. But the actual bug — a bad column reference, a type mismatch — is 20 transformations earlier.

Since nothing executes until an action is called, all errors appear at the action, not at their origin.

Fix for Problem 3

Debugging Confusion → explain()

```
df.filter(col("year") == 2024) \
  .groupBy("region") \
  .agg(sum("amount")) \
  .explain()
```

```
== Physical Plan ==
HashAggregate(keys=[region], functions=[sum(amount)])
+- Exchange hashpartitioning(region, 200)  ← SHUFFLE
   +- HashAggregate(...)
      +- FileScan parquet [region,amount]
         PushedFilters: [EqualTo(year,2024)]  ← PREDICATE PUSHDOWN ✓
         PartitionFilters: [year = 2024]      ← PARTITION PRUNING ✓
```

Cache vs Checkpoint

Strategy	What It Does	Keeps Lineage?	Use When
No cache	Recompute every time	Yes	Simple, cheap operations
`cache()` / `persist()`	Store in memory (or disk)	Yes — can recompute if evicted	Multiple actions on same DataFrame
`checkpoint()`	Save to reliable storage	**No — breaks lineage**	Long pipelines, fault tolerance

Benchmark (same dataset, same query)

No cache: **5,460 ms** → Cache: **1,074 ms** (5× faster) → Checkpoint: **742 ms** (7× faster)

Source: Spark in Action (Manning) — Ch. 16, Brazil dataset

Common Anti-Patterns

❌ Don't	✅ Do	Why
Multiple actions without cache	`cache()` before 2nd action	Each action = full recompute
`collect()` on large data	`take(n)`, `show()`, or `agg()`	collect() brings ALL data to driver → OOM
UDFs for simple logic	Built-in functions (`when`, `expr`)	UDFs block Catalyst optimization
Ignore `explain()`	Check plan for slow queries	Spot missing pushdowns, extra shuffles
Default 200 shuffle partitions	Tune `spark.sql.shuffle.partitions`	Match to cluster size and data volume

DEMO 3

Let's see it live

Caching · UDFs · `explain()` as a diagnostic tool · skew & salting (bonus)

Key Takeaways

1. Spark is fundamentally lazy

Transformations build the plan.

Actions execute it.

The Catalyst optimizer makes your plan better than you wrote it.

2. Partitioning is your data's architecture

In-memory: match partitions to your cluster.

On-disk: `partitionBy()` enables partition pruning.

Right-size, or pay the price.

3. Know the tools

`cache()` when reusing data.

`checkpoint()` long pipelines.

Always check `explain()` before optimizing blindly.

Thank You

Questions?

Demo code and notebooks: github.com/LizaMalinina/spark-talk

High Performance Spark (O'Reilly) · *Spark in Action* (Manning) · spark.apache.org

Illustrations by Alex Malinin

Liza Malinina